

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUA – PEC 2

Presentación

Esta PEC 2 y última prueba del curso recoge algunas de las buenas prácticas en el desarrollo y puesta en producción del software en arquitecturas distribuidas. Se trabajan algunos de los conceptos clave durante el desarrollo y puesta en producción, como son la entrega continua, la cultura DevOps y el despliegue utilizando contenedores.

Para elaborar la PEC se formulan cuatro ejercicios: 1) evaluar un aspecto a concretar de los principios fundamentales de la cultura DevOps; 2) razonar sobre los aspectos clave de las pruebas en la entrega continua de software de calidad; 3) evaluar uno de los aspectos clave de los contenedores y plataformas de orquestación; 4) explicar la construcción de un “pipeline” de entrega continua para probar y desplegar los microservicios implementados y testeados a las prácticas anteriores.

La PEC 2 abarca los contenidos de los libros "Microservices Patterns", "Continuous Delivery" y "The DevOps Handbook" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PEC 2 se trabaja las siguientes competencias del Grado en Ingeniería Informática:

- Saber construir aplicaciones informáticas mediante técnicas de desarrollo, integración y reutilización.
- Aplicación de las técnicas específicas de la Ingeniería del Software a las diferentes etapas del ciclo de vida de un proyecto.

Objetivos

Los objetivos de la PEC 2 son:

- Reconocer los factores de entrega, despliegue e integración continua.
- Interpretar la cultura ágil en el desarrollo de software.
- Entender los “pipelines” de entrega, despliegue e integración continua.
- Exponer argumentos sobre las ventajas y costes de los contenedores y plataformas de orquestación.

RECUERDA: Tal como se explica en el Plan docente de la asignatura, esta actividad debe resolverse de forma individual. Por otro lado, no está permitido el uso de herramientas de inteligencia artificial. En caso de detectar copias o uso de herramientas de IA, así como reutilización de soluciones de esta misma prueba de semestres anteriores, se penalizará la actividad con D como nota.

Para garantizar que has resuelto esta actividad de forma individual y sin uso de herramientas de IA ni reutilización de soluciones de semestres anteriores, te pedimos que en tu solución entregada añadas el siguiente compromiso de autorresponsabilidad:

Certifico que he realizado la PEC 2 de forma completamente individual y únicamente con la ayuda que el profesorado de esta asignatura considera oportuna, según las indicaciones explicadas en el documento “Originalidad en la evaluación” disponible en el aula. Entiendo que el trabajo no original y/o el empleo de IA generativa implicará que la actividad entregada no será corregida y que automáticamente se le asignará una calificación de D.

Compromiso de autorresponsabilidad

Escribe aquí el texto con el compromiso de autorresponsabilidad que consta en cursiva en la nota previa. **No escribirlo implica la no corrección de la PEC 2 y una calificación de D.**

Descripción de la PEC 2 a realizar

Ejercicio 1 (2,5 puntos)

Al inicio de esta actividad os hemos propuesto el siguiente debate: “Debate los objetivos básicos que persigue una cultura DevOps y los mecanismos, herramientas y/o estrategias para conseguirlos.”. En este ejercicio os pedimos que escojáis uno de los objetivos que hayáis debatido en el espacio “Debate de la Actividad 5” del aula para explicar cómo lo aplicarías al proyecto **Photo&Film4You**.

Nota: Debéis indicar la entrada del debate donde se explique el objetivo elegido.

Solución

En el debate del aula han salido muchos objetivos que persigue la cultura DevOps, entre otros:

- Aumentar la flexibilidad.
- Reducir o eliminar los conflictos entre los departamentos de operaciones y desarrollo.
- Automatizar al máximo los procesos de comunicación entre los departamentos de desarrollo y los de operaciones.
- Agilizar las entregas y la introducción de cambios en producción, consiguiendo una frecuencia de entregas más alta.
- Establecer un sistema de desarrollo y entrega más eficiente que permita reducir costes.
- Reducir los errores en entorno de producción y mejorar así la calidad del producto desarrollado
- Abreviar el ciclo de vida del desarrollo de software
- Proporcionar una entrega continua de software de alta calidad.
- Establecer mecanismos óptimos y fluidos de comunicación entre departamentos para aliviar y transparentar todo lo posible las colaboraciones entre todas las áreas.

Cualquiera de estos objetivos con una explicación clara, concreta, concisa, bien redactada y estructurada, y con un ejemplo de cómo lo haría para conseguirlo en el marco del proyecto **Photo&Film4You**, se considera como solución correcta al ejercicio.

Ejercicio 2 (2,5 puntos)

En la PRAC2 hicisteis la implementación de los microservicios *Course*, *Microcredentials* y *Notification* de **Photo&Film4You** mientras que en la PRAC3 desarrollasteis un conjunto de pruebas unitarias, de contrato e incluso de mutación para estos microservicios. Responded brevemente a las siguientes cuestiones:

1. La integración continua es la base para conseguir llevar software de calidad a producción de forma ágil. Indicad qué estrategia creéis que es la más óptima para el desarrollo, despliegue y puesta en producción de estos microservicios en un entorno de Continuous Delivery. Razonad la respuesta.
2. ¿Habéis seguido esta estrategia en el desarrollo de la asignatura? Razonad la respuesta.

Solución

1. Para conseguir la puesta en producción de los microservicios en un entorno de Continuous Delivery y Continuous Deployment es **imprescindible** tener una buena base de pruebas y estas **se deben realizar a medida que se van desarrollando las funcionalidades como parte integral del desarrollo de las mismas**, cada funcionalidad tiene que tener sus pruebas para que pueda ser desplegada automáticamente con confianza. Hacerlo así permite tener siempre una buena batería de pruebas ya que estas se pueden considerar como parte integrante del desarrollo de cada funcionalidad y sin ellas es imposible hacer despliegue continuo de funcionalidades.

Como habéis visto, la entrega continua de software se basa en la construcción, prueba y despliegue automatizado de software y uno de los mecanismos para garantizar que el software que se despliega no tenga errores es con una buena batería de test que se ejecuten como parte del *pipeline* de integración continua, que proporcionen una elevada cobertura y aborten la construcción y posterior despliegue en caso de errores. Estos tests no son los únicos que se tienen que pasar, pero sí son una parte importante en el momento de garantizar que las nuevas funcionalidades no introducen errores de regresión o de cualquier otro tipo.

También se puede ir un paso más allá y desarrollar, para cada funcionalidad, primero los tests que la prueban y después el código que la implementa. Esta técnica es conocida como *Test Driven Development* y es ampliamente utilizada en proyectos ágiles de desarrollo de software. Tener en cuenta que TDD puede ser una buena práctica, pero no es imprescindible para conseguir continuous delivery.

Nota: La solución se ha enfocado en la existencia de pruebas integrales mientras se realiza el desarrollo. Si habéis enfocado la respuesta de forma más global en la existencia de un pipeline de CI/CD y habéis explicado cómo lo habríais hecho, se ha considerado correcto.

2. **NO.** La estrategia adoptada en el proyecto fue hacer primero la construcción del software en la PRA2 y después, en la PRAC3 se desarrolló toda la parte asociada con el testing de la aplicación (tanto pruebas unitarias como pruebas de integración e incluso pruebas de validación arquitectónica). Estas pruebas son clave en el desarrollo de software de calidad, concretamente en la rápida entrega de nuevas funcionalidades ya que sin ellas no puedes garantizar que el despliegue continuo de las nuevas funcionalidades se produzca sin errores, por tanto, la parte de testing desarrollada en la PRAC3 no se puede desvincular del proceso de desarrollo.

Nota: Si habéis enfocado la respuesta en comentar que no se ha implementado un pipeline de CI/CD y lo habéis explicado correctamente, se ha considerado correcto.

Ejercicio 3 (2,5 puntos)

En la PRAC2, los microservicios se desplegaron usando un patrón de despliegue con un formato propio dependiente del lenguaje, mientras que la infraestructura de soporte se desplegaba con una estrategia basada en el uso de contenedores.

1. Analiza dos inconvenientes del patrón de despliegue basado con un formato propio dependiente del lenguaje que os podéis encontrar al usar este patrón para desplegar los microservicios en el proyecto **Photo&Film4You**.
2. Considerando un escenario de alta carga donde el microservicio de notificaciones requiere un escalado masivo debido a un aumento repentino en la emisión de microcredenciales, ¿cuál sería el patrón de despliegue óptimo? Razona la propuesta desde una perspectiva de arquitectura.

Solución

1. En la **PRAC2**, los microservicios se desplegaron como procesos Java (JAR) independientes. Alguno inconvenientes del uso de este patrón para desplegar los microservicios en el proyecto Photo&Film4You pueden ser:
 - **Falta de encapsulación del stack tecnológico:** Los administradores deben conocer los detalles específicos de cada servicio. Por ejemplo, para desplegar los microservicios de *Photo&Film4You*, es imprescindible que el host tenga instalada previamente la versión correcta de la máquina virtual de java. Si diferentes servicios requirieran versiones distintas de Java o librerías específicas del sistema operativo, se podrían producir conflictos de configuración y un aumento de errores durante el despliegue.
 - **No se pueden limitar los recursos consumidos:** Al ejecutarse como un proceso del sistema operativo, no hay forma de restringir cuánta CPU o memoria utiliza cada

microservicio. Aunque es cierto que, aunque el entorno de ejecución basado en la JVM de los microservicios de **Photo&Film4You** ofrece herramientas internas de limitación (por ejemplo, la limitación de la memoria heap consumida por cada proceso java), el **patrón de despliegue** en sí mismo carece de la capacidad de control y aislamiento que proporcionan las estrategias basadas en contenedores o las máquinas virtuales a nivel de sistema operativo.

2. Ante un escenario de escalado masivo y repentino por un pico de microcredenciales, el patrón más adecuado es **desplegar el microservicio como un servicio serverless**:

- **Elasticidad extrema:** A diferencia de los contenedores o máquinas virtuales que requieren pre-aprovisionar recursos, una estrategia basada en serverless ejecuta automáticamente tantas instancias del microservicio como sean necesarias para manejar la carga de eventos entrantes. En el caso de *Notification*, el sistema podría pasar de enviar 10 alertas a 10.000 de forma instantánea sin intervención manual.
- **Eficiencia de costes:** El servicio de notificaciones tiene un comportamiento basado en eventos (reacciona a mensajes de Kafka o creación de credenciales). Con el modelo *serverless*, **Photo&Film4You** solo pagaría por el tiempo de ejecución real de cada petición (milisegundos) y la memoria consumida, eliminando el coste de tener servidores encendidos e inactivos esperando la llegada de nuevos mensajes.
- **Reducción de carga administrativa:** Este modelo elimina por completo la necesidad de administrar sistemas operativos o runtimes. El equipo de desarrollo puede centrarse exclusivamente en la lógica de envío de la notificación, mientras que el proveedor de la nube gestiona el escalado y la alta disponibilidad del servicio.

También se considera correcto usar un patrón de **desplegar el microservicio en un orquestador de contenedores** que permita configurar un autoescalado horizontal de recursos en base a métricas o demanda de peticiones.

Consultar el capítulo “*Chapter 12. Deploying microservices*” de la bibliografía “*Microservices Patterns*” para ver el detalle de los patrones de despliegue seguidos.

Ejercicio 4 (2,5 puntos)

Tras la implementación de los microservicios en la PRAC2 y la definición de las pruebas en la PRAC3, el equipo de **Photo&Film4You** quiere automatizar su entrega mediante un pipeline de despliegue. Este pipeline se define como la manifestación automatizada del proceso que lleva el software desde el control de versiones hasta las manos del usuario.

La construcción del proyecto se lanza automáticamente a partir de la subida de nuevas funcionalidades al repositorio de código fuente por parte de los desarrolladores y se ha definido con estas cinco fases:

1. Fase de construcción: El pipeline se inicia automáticamente tras cada *commit* del desarrollador en el repositorio.
2. Fase de confirmación: Se realizan las validaciones técnicas iniciales y se crea el candidato a liberación.
3. Fase de aceptación: Se valida que el software cumple los requisitos funcionales y no funcionales del cliente.
4. Fase de testeo manual y de UAT: El software se despliega en un entorno similar a producción para pruebas exploratorias y validación de negocio.
5. Fase de despliegue a producción: El paso final de puesta en producción para entregar valor al usuario. En este caso el despliegue requiere una aprobación manual.

Responded brevemente a las siguientes cuestiones:

1. En la PRAC3 se definieron algunos de los cinco niveles de test típicos (unitarios, contrato, integración, componente y E2E). Sitúa cada uno de estos tipos de test en una de las fases del pipeline de despliegue si queremos obtener feedback y desestimar versiones que no pasen alguno de los tests lo más rápido posible. Razonad la respuesta.
2. ¿En qué fase deben generarse los ejecutables de cada microservicio y por qué es un antipatrón volver a compilarlo o recrearlo en pasos posteriores? Razonad la respuesta.
3. Si los responsables de Photo&Film4You deciden pasar de Continuous Delivery a Continuous Deployment, ¿cómo afectaría esto al flujo de trabajo entre las fases 4 y 5? Razonad la respuesta.

Solución

1. Para obtener feedback y desestimar versiones que no pasen alguno de los tests lo más rápido posible, los tests deben organizarse por velocidad y aislamiento:

- a. **Fase de Confirmación:** Aquí se deben situar las pruebas unitarias y de contrato (lado consumidor) ya que los dos tipos son muy rápidos (milisegundos) y no requieren infraestructura. Ejecutando los tests en esta etapa se detectarán errores técnicos inmediatamente y el desarrollador no perderá el contexto de su cambio.
 - b. **Fase de aceptación:** Aquí podemos situar los tests de integración (*test containers*), los tests de componente y los test E2E ya que estos tests implican levantar infraestructura como la base de datos (PostgreSQL), las colas de mensajería (Kafka) o incluso múltiples microservicios reales, lo que los hace más lentos y costosos en recursos. Al situarlos en la fase de aceptación, solo se ejecutan si el código ha pasado los tests “rápidos” de la fase de confirmación, ahorrando tiempo y computación
2. Los ejecutables de cada microservicio deben generarse exclusivamente en el paso de confirmación ya que solo se construyen los binarios una vez y se tienen que usar estos mismos en todas las etapas del pipeline. Reutilizar el mismo artefacto generado en el paso de confirmación durante todo el resto del pipeline garantiza que el software que se validó en las pruebas de aceptación es exactamente el mismo que se entrega a producción, asegurando la integridad del proceso, si se reconstruye en fases posteriores no se puede asegurar.
3. Pasar de Continuous Delivery a Continuous Deployment implica eliminar la aprobación manual para el paso a producción:
 - a. En Continuous Delivery (estrategia actual de la pregunta), el paso de la fase testeo manual y UAT al paso de despliegue en producción requiere una aprobación manual. El pipeline "empuja" el código hasta QA, pero el paso a producción NO es una decisión automática.
 - b. En Continuous Deployment, el paso de la fase de pruebas manuales y UAT al paso de despliegue en producción se automatiza por completo. Si el candidato a liberación supera todas las validaciones de las fases de confirmación, aceptación y pruebas manuales y UAT, se despliega directamente en producción sin intervención humana. Es necesario sustituir la fase de pruebas manuales y UAT por pruebas automatizadas y trasladar las pruebas manuales exploratorias o UAT a producción utilizando técnicas como Canary Releases o Feature Toggles. Todo ello requiere una confianza absoluta en la batería de pruebas de los pasos anteriores para evitar impactos negativos en los usuarios.

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson) Chapter 12. Introduction.
- Libro "Microservices Patterns "(Chris Richardson) Chapter 12 "Deploying microservices".
- Libro "Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation" (Jez Humble & Dave Farley) Parte I: Chapters 1-10.
- Libro "The DevOps Handbook" (Patrick Debois; John Willis; Jez Humble; Gene Kim): Chapters 1 - 4.

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. **Esto incluye la reutilización de soluciones de esta misma prueba de semestres anteriores**. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PEC2.pdf*.

Este documento se entregará en el espacio "Entrega de la PEC2" del aula antes de las **23:59 horas del día 22 de junio de 2026**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación de la PEC 2

	Excelente (10 puntos)	Notable (7 puntos)	Suficiente (5 puntos)	Insuficiente (2,5 puntos)	Sin respuesta / Todo mal (0 puntos)
Ej. 1	Explicación clara, concreta, concisa, bien redactada y estructurada. Ejemplo correcto. (2,5 puntos)	Explicación correcta pero poco concisa y/o mal redactada. Ejemplo correcto. (2 puntos)	Explicación correcta o Ejemplo correcto. (1,5 puntos)	Explicación razonada incorrecto y Ejemplo incorrecto. (0,75 puntos)	Sin respuesta, no hay ninguna justificación. (0 puntos)
Ej. 2	Respuestas correctas y razonadas. (2,5 puntos)	Respuestas correctas con algún error leve en el razonamiento. (2 puntos)	Respuestas correctas con alguna de ellas sin razonamiento o mal razonada. (1,5 puntos)	Respuestas incorrectas, mal razonadas o sin razonamiento. (0,75 puntos)	Todo incorrecto o sin respuesta. (0 puntos)
Ej. 3	Respuestas correctas y razonadas. (2,5 puntos)	Respuestas correctas con algún error leve en el razonamiento. (2 puntos)	Respuestas correctas con alguna de ellas sin razonamiento o mal razonada. (1,5 puntos)	Respuestas incorrectas, mal razonadas o sin razonamiento. (0,75 puntos)	Todo incorrecto o sin respuesta. (0 puntos)
Ej. 4	Respuestas correctas y razonadas. (2,5 puntos)	Respuestas correctas con algún error leve en el razonamiento. (2 puntos)	Respuestas correctas con alguna de ellas sin razonamiento o mal razonada. (1,5 puntos)	Respuestas incorrectas, mal razonadas o sin razonamiento. (0,75 puntos)	Todo incorrecto o sin respuesta. (0 puntos)